

COMPOSE -- DEV STAGE

Instrucciones

Vamos a dockerizar el blog personal que **Laura hizo con Lidia en la asignatura de Backend de segundo de DAW**. Utiliza el ZIP de los materiales como punto de partida.

La práctica se desarrolla en **tres etapas**: un entorno de desarrollo, [Prod Stage 1 con Unicorn](#) y [Prod Stage 2 con Nginx](#). Cada etapa amplía el trabajo de la anterior.

Objetivo de esta etapa

Localiza los componentes de la práctica y reúne lo necesario para integrarlos en un entorno de desarrollo con Docker Compose. El alcance de esta entrega es el de la captura: runserver, recarga automática, servicio de archivos estáticos y django-browser-reload. Unicorn y Nginx se incorporarán en las siguientes etapas.

Qué debes localizar en el ZIP

- `manage.py` y el paquete `personalblog/`, con la configuración, las rutas y el punto de entrada WSGI de Django.
- Las aplicaciones `aboutme`, `posts`, `users` y `comments`: presentación del blog, publicaciones, usuarios y comentarios.
- `db.sqlite3`, las migraciones y `media/`, donde se guardan las imágenes subidas.
- Las plantillas, los recursos de `static/` y la aplicación `theme`. Tailwind se configura y compila desde `theme/static_src/`; el CSS generado está en `theme/static/css/dist/`.
- Las dependencias Python y las herramientas Node/npm. El Pipfile indica Python 3.13, pero su lista de paquetes está vacía: identifica las dependencias que utiliza el código, como Django, `django-tailwind`, `django-browser-reload` y Pillow.

Trabajo a realizar

1. Prepara las dependencias y los archivos Dockerfile y Compose para el entorno de desarrollo. Adapta la ruta de Windows de `NPM_BIN_PATH` en `personalblog/settings.py` al ejecutable de npm disponible en el contenedor.
2. Arranca Django con `python manage.py runserver 0.0.0.0:8000` y permite acceder al blog desde el navegador del equipo.
3. Monta el código para poder editarlo desde el equipo y comprobar la recarga automática. Mantén localizados y disponibles la base de datos SQLite y los archivos de `media/`.

4. Comprueba el servicio de estáticos en desarrollo: CSS, JavaScript e imágenes. Prepara las herramientas de Tailwind de `theme/static_src/` para poder recompilar los estilos durante el trabajo.
5. Integra y comprueba `django-browser-reload`. Revisa sus entradas en `INSTALLED_APPS`, `MIDDLEWARE` y la ruta `__reload__`; ya aparecen en la configuración del proyecto original.

Este entorno se utiliza localmente para desarrollar. `runserver` no es el servidor que emplearemos en las etapas de producción.

Compose – dev deploy

● `python manage.py runserver` → Pensado para desarrollor ***SOLO*** localmente

→ Recarga automática

→ **Sirve ficheros estáticos**

→ `django browser reload` para recarga automática ante cambios.

CONTRAS:

→ solo para desarrollo

→ servidor minimalista e inseguro

→ no sirve para alta carga ni múltiples procesos.

Referencia de Dev Stage. [Ver diapositiva 115 ↗](#)

Entrega

Adjunta el enlace al repositorio de GitHub y un `README.md` con el inventario de componentes, las dependencias identificadas, los pasos de arranque y capturas del blog, de los estáticos y de la recarga funcionando. Este repositorio será la base de las dos siguientes etapas.

Materiales de la tarea

Los archivos adjuntos se descargan por separado desde la página de esta tarea.



[laura-personal_blog-main.zip](#)

Blog personal de Laura · proyecto base para las tres etapas · ZIP, 2,6 MB

